ASIC Onboarding Program

Program Guide, Learning Resources and Lab Roadmap

RTL Design, Functional Verification, Physical Design and Architecture Modeling for RISC-V Processor Teams

Kerim Turak

October 24, 2026

Program objective: Give every new engineer the same foundation in SystemVerilog RTL design, verification and implementation by building a small set of blocks, verifying those same blocks, and then synthesizing and timing them — and then prepare each engineer for their role in the team (RTL, design verification or architecture modeling) on top of that foundation.

Contents

1 Program Overview **3**
1.1 Why the Program Is Organized by Phases . . . 3
1.2 Program Map . . . 4
1.3 Duration and Weekly Effort . . . 4

2 Labs **5**
2.1 Lab List . . . 5
2.2 How the Designs Flow Through the Labs . . . 6
2.3 What You Will Build . . . 6
2.4 Rules Between Phases . . . 7
2.5 Tracks . . . 7

3 Test Types Used in the Program **8**

4 Training Repository **9**

5 Week-by-Week Roadmap **10**

6 Primary Books **11**
6.1 Computer Architecture . . . 11
6.2 SystemVerilog Design and Verification . . . 11
6.3 Optional References . . . 11

7 Video Lectures **12**
7.1 Primary: Onur Mutlu, Digital Design and Computer Architecture . . . 12
7.2 Secondary: David Wentzlaff, Computer Architecture (Princeton) . . . 12

7.3 Verification: Verification Academy and UVM Course 12

8 Practice Resources and Specifications **12**

8.1 HDLBits 12

8.2 RISC-V Specifications 12

8.3 Open-Source Reference Projects 13

9 Development Tools **13**

9.1 Open-Source Tools and Their Synopsys Equivalents 14

9.2 Other Tools 15

10 Weekly Study Requirements **15**

10.1 Reading and Lectures 15

10.2 Weekly Technical Summary 15

10.3 Weekly Review 15

11 Submission Requirements **15**

12 Assessment **16**

13 Completion Criteria **16**

14 Final Objective **16**

1 Program Overview

This document defines how the onboarding program is organized: the labs and their order, the shared repository every engineer works in, the books, lectures and tools, and how progress is reviewed and assessed. It is written for engineers with limited or no experience in processor design, and for the coordinators who run the program.

1.1 Why the Program Is Organized by Phases

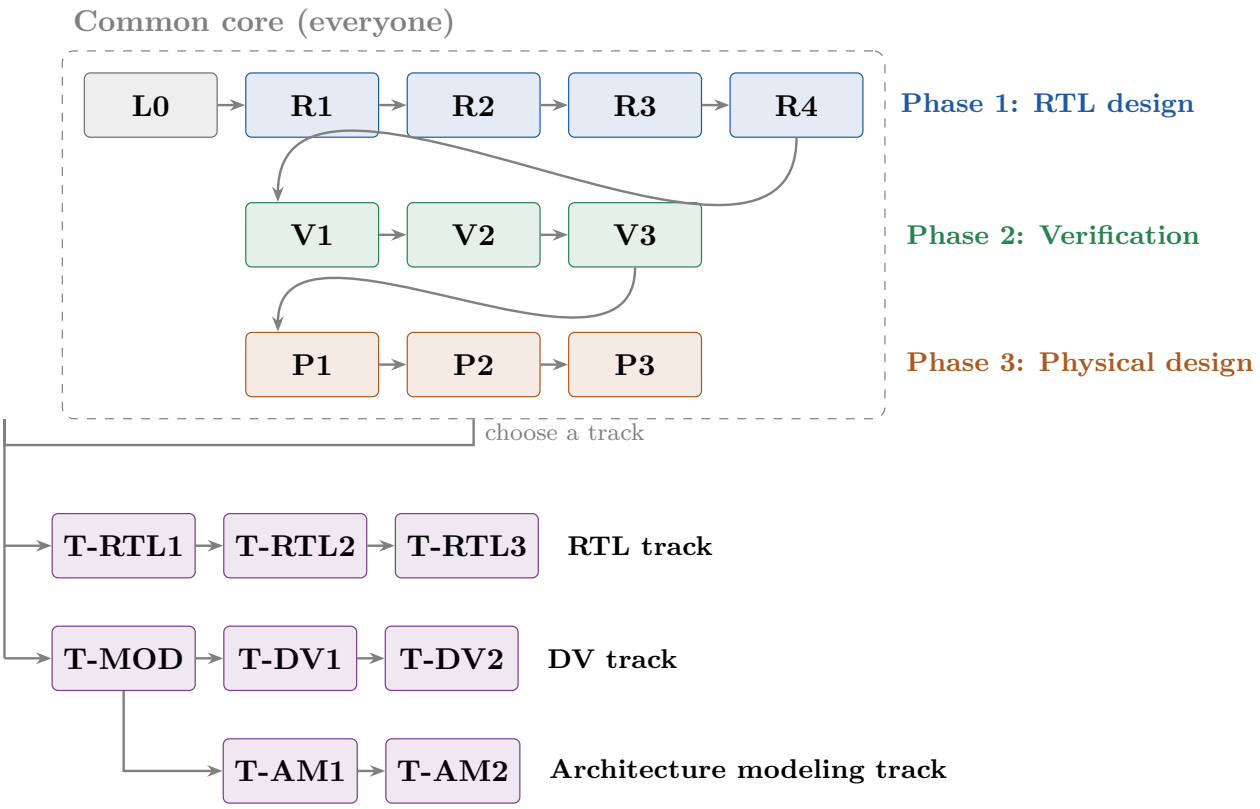
In a real ASIC team a block passes through the hands of several people: a designer writes the RTL, a verification engineer proves it correct, and an implementation engineer turns it into gates and closes timing. Each of them needs to understand the others' work. An earlier version of this program asked for all of these activities in every lab, which made each lab very large and made the labs feel disconnected.

The program is therefore organized in **phases on the same designs**:

1. **Phase 1 — RTL design.** You design four blocks of increasing complexity: a sequential multiplier, a synchronous FIFO, an arbitrated memory and a data cache. Each comes with a small directed testbench, enough for the designer to trust the basic behavior.
2. **Phase 2 — Verification.** You return to your own four designs with the verification engineer's view: random stimulus, scoreboards, assertions, classes, constrained randomization and functional coverage. You will find bugs in code you thought was correct — that is the point.
3. **Phase 3 — Physical design and PPA.** You synthesize, gate-level simulate and time the same verified designs, read area and timing reports, and connect every line of a report back to the RTL you wrote.

Because each phase revisits designs you already know, you can focus on the new skill instead of on a new specification. After the three phases, the **common core**, engineers continue in a **track** for their role.

1.2 Program Map



1.3 Duration and Weekly Effort

Part	Weeks	Who
Common core: L0 and Phases 1–3	approx. 17	Everyone
RTL track (T-RTL1–3)	approx. 5.5	RTL / design engineers
DV track (T-MOD, T-DV1–2)	approx. 6.5	Verification engineers
Arch. modeling track (T-MOD, T-AM1–2)	approx. 5	Architecture engineers

The expected commitment is 15–20 hours per week. Durations may be adjusted to each engineer's background: an engineer who already writes production RTL may compress Phase 1, while a software engineer moving into architecture modeling may need more time in Phase 1 and less in Phase 3. The coordinator records every adjustment.

Activity	Hours per week
Lab work (design, verification, implementation)	8–11
Book reading and technical study	3–4
Video lectures	2–3
Weekly summary and review preparation	1–2
Total	15–20

Practical work takes priority over passive video consumption. Engineers demonstrate understanding

through working RTL, verification results, reports they can explain, and technical discussions in the reviews.

2 Labs

2.1 Lab List

Lab	Title	Main skills	Weeks
Common core: setup			
L0	Repository, Tools and Flow	Repository layout, file lists, Make, Git workflow, hello-world IP	0.5
Common core: Phase 1 RTL design			
R1	Sequential Multiplier	SystemVerilog RTL constructs, FSM, valid/ready, simulation–synthesis-safe coding rules, lint	1.5
R2	Synchronous FIFO	Parameterization, scheduling regions and races	1
R3	Arbitrated Memory	Interfaces and modports, packed/unpacked arrays, round-robin arbitration	1.5
R4	Data Cache	Cache micro-architecture, controller FSM, performance counters, hit-rate prediction	2
Common core: Phase 2 verification			
V1	Self-Checking Testbench and Assertions	Driver/monitor/scoreboard, randomization, plusargs, regression, SVA, testplans	2
V2	Verifying a Shared Resource	Virtual interfaces, dynamic arrays and queues, fork/join, protocol assertions	1.5
V3	Constrained-Random Verification of the Cache	Classes, constraints, functional coverage, coverage closure	2
Common core: Phase 3 physical design			
P1	Logic Synthesis and Gate-Level Simulation	Yosys or Fusion Compiler, standard cells, GLS, simulation–synthesis mismatches	1.5
P2	Static Timing Analysis	SDC, timing reports, critical paths, Fmax, I/O constraints	1.5
P3	Area, Timing and Scaling of Larger Blocks	Parameter sweeps, memories, PPA trade-offs	1.5
RTL track			
T-RTL1	Clock Domain Crossing Library	Synchronizers, asynchronous FIFO, multi-clock verification, CDC constraints	2
T-RTL2	Multi-Clock Integration	CDC integration, crossing report, two-clock STA	1.5
T-RTL3	Timing-Driven RTL	Pipelining to a frequency target, retiming, skid buffers	2
DV and architecture modeling tracks			
T-MOD	Transaction-Level Cache Model	Object-oriented SystemVerilog, reference models	1.5

Lab	Title	Main skills	Weeks
T-DV1	Layered Class-Based Testbench	Generator/driver/monitor/scoreboard classes, model-based checking	2
T-DV2	UVM Environment for the Cache	Agents, sequences, factory, configuration, phases	3
T-AM1	Design-Space Exploration	Trace-driven simulation, model–RTL correlation	1.5
T-AM2	Cycle-Approximate Timing Model	Timing models, calibration, cycle correlation, what-if studies	2

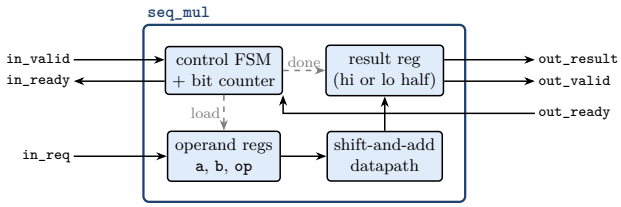
2.2 How the Designs Flow Through the Labs

Every block is touched by every phase. The table shows where each block is created, verified and implemented, and which track reuses it.

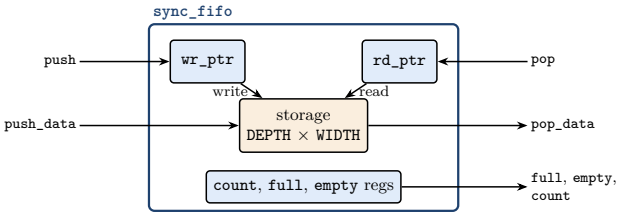
Block	Designed	Verified	Implemented	Reused in tracks
`seq_mul`, `comb_mul`	R1	V1	P1, P2	T-RTL3 (pipelined version)
`sync_fifo`	R2	V1	P1, P2	—
`arb_mem`, `mem_if`	R3	V2	P3	Memory of the cache in every later lab
`dcache`	R4	V3	P3	T-RTL2, T-MOD, T-DV1, T-DV2, T-AM1, T-AM2
CDC library	T-RTL1	T-RTL1	T-RTL1	T-RTL2
Cache model	T-MOD	T-MOD	—	T-DV1, T-DV2, T-AM1, T-AM2

2.3 What You Will Build

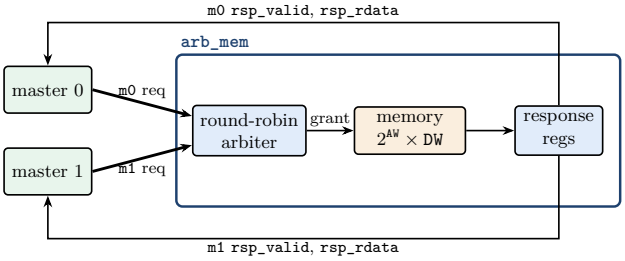
Figure 1 shows the four blocks of the common core at the level of detail you will draw yourself in each block's `doc/design.md`: the ports at the boundary, and inside, the boxes you will write RTL for. Blue boxes are the design (the DUT), orange boxes are storage arrays, green boxes belong to the testbench. Every lab repeats its own block diagram in more detail.



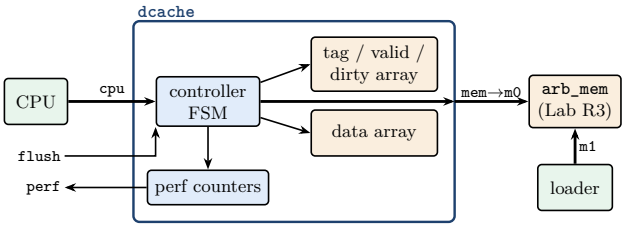
(a) Lab R1: sequential multiplier



(b) Lab R2: synchronous FIFO



(c) Lab R3: memory shared by two `mem_if` masters



(d) Lab R4: data cache in front of `arb_mem`

Figure 1: The four common-core blocks. Each block is designed in Phase 1, verified in Phase 2 and synthesized and timed in Phase 3. The cache in (d) does not copy the memory of (c): it instantiates it through a file list.

The tracks add three more designs on top of these: a CDC library and a two-clock wrapper of the cache (`dcache_cdc_top`, RTL track), a pipelined multiplier `seq_mul_pipe` (RTL track), and a transaction-level model of the cache (`cache_model_pkg`, DV and performance tracks). Their block diagrams are in the track labs.

2.4 Rules Between Phases

- **A phase does not pre-empt the next one.** Phase 1 labs do not ask for assertions or synthesis; Phase 2 labs do not ask for timing reports. Each lab says explicitly which later lab covers a topic it leaves out.
- **Designs are not rewritten between phases.** When verification finds a bug, the RTL is fixed in place, with a commit that names the failing test. When timing analysis shows a problem, the RTL change is measured and explained. The history of each block is visible in Git.
- **Nothing breaks.** After every lab, all earlier blocks must still lint, simulate and (from Phase 3 on) synthesize with the shared flow (`make check-all`).

2.5 Tracks

The track is agreed with the team lead before the common core ends. An engineer may complete a second track later.

- **RTL track.** Clock domain crossing, multi-clock integration and timing-driven design — the issues that dominate real block design after functional correctness.
- **DV track.** A transaction-level reference model, a layered class-based testbench built by hand, and then the same testbench in UVM, the industry methodology.
- **Architecture modeling track.** The same reference model, used for design-space exploration and extended into a cycle-approximate timing model that is calibrated and correlated against the RTL.

The architecture modeling track works with SystemVerilog transaction-level models of the blocks built in the core. It is not the team's performance modeling: the C++ performance models of the processor (Sniper and similar simulators) are owned by the performance team and are outside this program. The methods are the same — calibration against RTL, correlation, what-if studies — and an engineer who completes this track can read and question those models.

The DV and architecture modeling tracks share their first lab (T-MOD): the model a verification engineer uses as a golden reference is the same model a performance engineer uses to explore the design space. Working on one model from both sides is how these two teams cooperate in practice.

3 Test Types Used in the Program

The labs use the names of tests the way verification teams use them. Each name says *why* a test is run and *what a pass proves*; a test name without that meaning is only a label. This table is the shared reference; later labs focus on what each test checks for that block.

Test	**What it is and what a pass proves**	**First used**
Smoke test	A short, directed test of the most basic behavior: reset, one or a few normal operations, compare with the expected result. It answers "is the block alive and is the flow working?" in seconds. A pass proves very little about correctness; a failure means nothing else is worth running. The name comes from hardware bring-up: switch on, see whether smoke comes out.	L0 (`counter`), every R lab
Sanity test	Often used as a synonym of smoke test. In this program: the smoke test of every block run together (`make check-all`) to check that a change did not break anything that used to work.	L0
Directed test	A hand-written sequence of stimulus with a known expected result, aimed at one feature ("fill the FIFO, then drain it"). Easy to debug, but only checks what its author thought of.	R1–R4
Corner-case test	A directed test at the edges of the specification: maximum/minimum operands, full and empty FIFO, push and pop in the same cycle, both masters requesting in the same cycle, eviction of a dirty line. Most bugs live here.	R1–R4
Exhaustive test	Every possible input combination is applied (possible only for small configurations, e.g. the multiplier with `MUL_XLEN=8`: $3 \cdot 2^{16}$ operations). A pass is a proof for that configuration.	R1
Random test	Stimulus drawn from `$urandom` with a printed seed, thousands of transactions, checked by a reference model and a scoreboard. Finds bugs nobody thought of; the seed makes a failure reproducible.	V1
Constrained-random test	Random stimulus whose legal ranges and distributions are described with class constraints (`rand`, `constraint`, `dist`), so that the generator spends its time on interesting cases.	V3
Back-pressure / stress test	The consumer randomly de-asserts `ready` (back-pressure) or both sides run at maximum rate (stress), to exercise every handshake stall and every full/empty transition.	V1, V2

Test	What it is and what a pass proves	First used
Negative / error-injection test	Deliberately illegal or broken behavior (protocol violation, a bug inserted in the RTL) to prove that the checkers — scoreboard and assertions — actually fire. A checker that never fails proves nothing.	V1 (bug hunt), V2, T-DV1
Regression	The whole set of tests × seeds × parameter sets, run by one command (`make regress`) with a pass/fail summary. Run before every merge; a *nightly regression* runs more seeds every night.	V1
Assertion (SVA) checks	Properties checked on every clock edge during simulation (or proven by a formal tool). They catch a protocol or design-rule violation at the cycle it happens, instead of when a wrong value reaches the scoreboard.	V1, V2
Coverage closure	Not a test but a completion criterion: functional coverage (covergroups, cover properties) shows which features the tests actually exercised; the regression is extended until the coverage plan is met.	V3
Gate-level simulation (GLS)	The verification testbench run on the synthesized netlist instead of the RTL. Finds simulation–synthesis mismatches, missing resets and X problems that RTL simulation hides.	P1
Model correlation test	The same trace run on the transaction-level model and on the RTL; the counters (hits, misses, write-backs, cycles) must match exactly or within a stated error. Proves that the model can be trusted for exploration.	T-AM1, T-AM2

NOTE | How the names grow through the program.

In Phase 1 the designer writes a *smoke* test plus *directed* and *corner-case* tests (and an *exhaustive* one where it is cheap). In Phase 2 the verification engineer adds *random* and *constrained-random* tests, *back-pressure* and *error-injection* tests, *assertions* and *coverage*, all collected in a *regression*. In Phase 3 the regression is re-run on the netlist (*GLS*). The smoke test is never deleted: it remains the first test of every regression and the test of `make check-all`.

4 Training Repository

Every engineer works in their own copy of the training repository. Its layout follows the conventions of large open-source ASIC projects — OpenTitan, Ibex, CVA6 and the PULP `common_cells` library — simplified for training. Lab L0 explains every directory and builds the shared flow; the overview is given here.

```
asic-onboarding/
+-- README.md, env.sh, Makefile, .gitignore
+-- doc/                 weekly reports, review notes
+-- flow/                shared build infrastructure (never design-specific)
|   +-- mk/              common.mk, lint.mk, sim_verilator.mk, regress.mk, syn.mk, sta.
    mk
|   +-- syn/  sta/       generic Yosys and OpenSTA scripts
|   +-- scripts/         tool checks, file-list expansion, sweeps
+-- lib/                 standard-cell libraries (provided, not committed)
+-- hw/
|   +-- ip/<block>/      one directory per block:
|   |                    rtl/ tb/ dv/ syn/ lint/ doc/ README.md Makefile
|   +-- top/<top>/       integration tops (RTL track)
+-- dv/                  shared verification code: utilities, VIP, UVM agents,
|   +-- model/           SystemVerilog transaction-level models, traces, exploration
```

```
+-- build/                all generated output (never committed)
```

Inside a block, each phase owns a directory: Phase 1 writes `rtl/`, `tb/` and `doc/design.md`; Phase 2 writes `dv/` and `doc/verification.md`; Phase 3 writes `syn/` and `doc/ppa.md`. The directory tree of a finished block therefore shows its whole life cycle — exactly like a block in a production repository.

Reference solutions. The program repository contains a `solution/` folder: a complete, finished copy of the training repository (`solution/asic-onboarding/`) with the RTL, testbenches, verification environments, file lists, make flow, synthesis and timing scripts and models of every lab, and an index (`solution/README.md`) that lists, for each lab, the files that solve it and the commands that reproduce its results. The solution folder is for coordinators and reviewers only: it is not part of the template or of the Overleaf projects given to engineers, and engineers must not copy from it. During a review the coordinator may compare an engineer's results (cycle counts, coverage, area, slack) against the solution's.

5 Week-by-Week Roadmap

The roadmap below aligns the labs with reading and lectures. The labs are block-level; the reading track continues the processor architecture topics (ISA, pipelining, hazards, exceptions) in parallel, so that by the end of the core the engineer understands both the blocks and the processor they will sit in. The coordinator assigns exact book sections and lecture links before each week.

Week	Lab	Architecture and theory reading	Resources
1	L0, R1	Digital logic, combinational and sequential circuits, FSMs	Harris & Harris ch. 1–3; Mutlu intro lectures; HDLBits basics
2	R1	Arithmetic circuits, multipliers; RISC-V M extension	Harris & Harris ch. 5; RISC-V unprivileged spec (M); *SystemVerilog for Design*
3	R2	Timing of sequential logic; HDL simulation semantics	Harris & Harris ch. 3; IEEE 1800 clause 4; Cummings NBA paper
4	R3	Memory arrays; buses and arbitration	Harris & Harris ch. 5 (memory arrays); *SystemVerilog for Design* (interfaces)
5–6	R4	Memory hierarchy and caches	Harris & Harris ch. 8; Mutlu cache lectures; Wentzlaff cache modules
7	R4 / V1	RISC-V ISA and instruction formats	RISC-V unprivileged spec; Verification Academy testbench basics
8	V1	Single-cycle processor organization	Harris & Harris ch. 7; Verification Academy SVA
9	V2	Pipelining and pipeline registers	Mutlu pipelining lectures; Wentzlaff pipelining
10	V2 / V3	Data and control hazards, forwarding	Harris & Harris ch. 7; Spear & Tumbush (classes)
11	V3	Exceptions, interrupts, CSRs	RISC-V privileged spec; Verification Academy coverage
12	V3 / P1	Processor verification: ISA models, co-simulation, riscv-dv	Spike, riscv-tests, riscv-dv documentation
13	P1	ASIC design flow; standard cells and libraries	Rabaey (optional); Yosys manual or Fusion Compiler user guide; SkyWater PDK docs

Week	Lab	Architecture and theory reading	Resources
14–15	P2	Static timing, setup and hold, clocks	OpenSTA documentation or PrimeTime user guide; Harris & Harris timing sections
16–17	P3	SRAMs, area and power; branch prediction overview	Wentzlaff memory and branch prediction modules
18+	Track	Superscalar, out-of-order, coherence (overview); track-specific reading	Wentzlaff advanced modules; track lab references

Important: the advanced architecture topics (superscalar, out-of-order, coherence) are introduced at overview level. They do not imply that an engineer becomes independently productive in a production out-of-order CPU after a week of reading. Depth is adjusted to the team's processor.

6 Primary Books

Study the assigned sections; do not read every book cover to cover.

6.1 Computer Architecture

Digital Design and Computer Architecture: RISC-V Edition, Sarah Harris and David Harris. The primary textbook of the program, also used in Onur Mutlu's course, so reading and lectures are closely aligned. Topics: digital logic, combinational and sequential circuits, FSMs, arithmetic and memory arrays, RISC-V instruction execution, single-cycle and pipelined processors, hazards, memory hierarchy.

6.2 SystemVerilog Design and Verification

- **SystemVerilog for Design**, Stuart Sutherland, Simon Davidmann and Peter Flake — data types, procedural blocks, enums and structs, packages, interfaces, synthesizable RTL. Used in Phase 1.
- **SystemVerilog for Verification**, Chris Spear and Greg Tumbush — testbench structure, randomization, classes, coverage, OOP testbench guidelines. Used in Phase 2 and the DV track.
- **SystemVerilog Assertions Handbook** (Cohen et al.) or the assertion chapters of IEEE 1800 — optional depth for V1–V2.

6.3 Optional References

- *Computer Organization and Design: RISC-V Edition*, Patterson and Hennessy (alternative architecture text).
- *Computer Architecture: A Quantitative Approach*, Hennessy and Patterson (architecture modeling track).
- *Digital Integrated Circuits*, Rabaey, Chandrakasan and Nikolić (circuit-level depth for Phase 3).

7 Video Lectures

7.1 Primary: Onur Mutlu, Digital Design and Computer Architecture

- Lecture videos and course materials: https://people.inf.ethz.ch/omutlu/lecture-videos.html
- YouTube: https://www.youtube.com/@OnurMutluLectures

The coordinator selects the semester and maintains a lecture-to-week mapping in `doc/lecture-map.md` of the training template. Watch the lectures that support the current week; the goal is not to watch every lecture available.

7.2 Secondary: David Wentzlaff, Computer Architecture (Princeton)

https://www.coursera.org/learn/comparch. A deeper treatment of micro-architecture and memory systems for an advanced audience: pipelining in depth, caches, memory management, branch prediction, superscalar and out-of-order execution, coherence. Use the pipelining modules alongside weeks 9–10, the cache modules alongside R4 and the architecture modeling track, and the advanced modules in the tracks. Check course availability before the program starts.

7.3 Verification: Verification Academy and UVM Course

- Verification Academy, https://verificationacademy.com/: testbench fundamentals (V1), SystemVerilog Assertions (V1–V2), OOP, constrained random and coverage (V3, T-MOD), UVM (T-DV2).
- *Design Verification with SystemVerilog/UVM* (Udemy), https://www.udemy.com/course/design-verification-with-systemverilog-uvm/: the SystemVerilog sections alongside Phase 2, the UVM sections in T-DV2 only.

UVM is introduced only after an engineer can build and explain a testbench with a driver, monitors, a reference model and a scoreboard by hand (V1–V3, T-DV1). Each UVM concept is then a replacement for something the engineer already wrote.

8 Practice Resources and Specifications

8.1 HDLBits

https://hdlbits.01xz.net/wiki/Main_Page. Interactive exercises for weeks 1–3: Verilog fundamentals, combinational logic, sequential logic, multiplexers, arithmetic, counters and FSMs. Passing an exercise is not enough: you must be able to say whether the circuit you described is combinational or sequential and draw it.

8.2 RISC-V Specifications

https://riscv.org/ — the unprivileged specification (RV32I, M extension, instruction formats) and the privileged specification (CSRs, exceptions, interrupts). The specifications are the authority on architectural behavior; textbooks and lectures are learning aids.

8.3 Open-Source Reference Projects

These projects are used throughout the program as examples of how industrial teams organize RTL, verification and implementation. Engineers are not expected to understand them completely; the coordinator assigns specific files to read and explain.

Project	What it is	Look at it for
Ibex (lowRISC)	Small in-order RV32 core	`rtl/`, `dv/uvm/`, `syn/` (Yosys + OpenSTA flow; compare with your Fusion Compiler + PrimeTime scripts), `lint/` waivers
OpenTitan (lowRISC)	Silicon root-of-trust SoC	Per-IP layout `hw/ip/<ip>/{rtl,dv,doc,data,lint}`, testplans, shared DV library `hw/dv/sv`
CVA6 (OpenHW Group)	Application-class RV64 core	`core/`, `verif/`, `pd/synth`, `perf-model/`, file lists with `${CVA6_REPO_DIR}`
`common_cells` (PULP)	Library of reusable primitives	FIFOs, arbiters, CDC FIFOs, formal property files — compare with your own R2, R3 and T-RTL1 blocks

9 Development Tools

The labs are written for open-source tools, so that every result is reproducible on any machine, including a laptop. The team works on the **Synopsys cloud**, where the industrial equivalents of these tools are installed and licensed. **If you have the Synopsys tools, use them**: they are the tools you will use in production, and they report more than their open-source counterparts. Every lab keeps the open-source commands and adds a blue *On the Synopsys cloud* box with the equivalent commands. The shared make flow (Lab L0) selects the tool with one variable, so the targets and the directory layout do not change.

9.1 Open-Source Tools and Their Synopsys Equivalents

Purpose	Open source (lab baseline)	Synopsys cloud equivalent	Make variable, first lab, remarks
RTL lint	Verilator `--lint-only`	SpyGlass Lint (`sg_shell`, goal `lint/lint_rtl`) or VC SpyGlass Lint in VC Static (`vc_static_shell`)	`LINT_TOOL`: `verilator`, `spyglass`, `vcst`, L0. SpyGlass is the industry sign-off lint; its rule set is much larger than Verilator's
RTL simulation	Verilator 5.x	VCS (`vcs -sverilog`)	`SIM`: `verilator`, `vcs`, L0. VCS is four-state (shows X), and fully supports constrained random, covergroups and UVM
Waveforms	GTKWave (VCD/FST)	Verdi (FSDB, `verdi -ssf`)	`make waves`, L0. Verdi also shows the source, the schematic and the FSM of the DUT
Functional and code coverage	Verilator coverage (limited)	VCS `-cm` + URG / Verdi Coverage	`COV=1`, `make cov`, V3
Formal property checking	— (SymbiYosys, optional)	VC Formal (`vcf` in VC Static)	Optional in V1–V2: prove the SVA you wrote
SV → Verilog conversion	sv2v	not needed	Synopsys tools read SystemVerilog directly
Logic synthesis	Yosys + ABC	Fusion Compiler (`fc_shell`); Design Compiler (`dc_shell`) where licensed	`SYN_TOOL`: `yosys`, `fc`, `dc`, P1
Equivalence checking	—	Formality (`fm_shell`)	Optional in P1: prove RTL ≡ netlist
Gate-level simulation	Verilator (netlist + cell models)	VCS (netlist + cell Verilog models)	`make gls`, P1
Static timing analysis	OpenSTA	PrimeTime (`pt_shell`)	`STA_TOOL`: `opensta`, `pt`, P2. PrimeTime is the sign-off STA tool
Power analysis	—	PrimePower (in `pt_shell`)	Optional in P3
Clock domain crossing	manual review + simulation	SpyGlass CDC / VC SpyGlass CDC	T-RTL1, T-RTL2: structural CDC check of every crossing
UVM	Verilator (partial) or EDA Playground	VCS `-ntb_opts uvm-1.2`	`SIM=vcs UVM=1`, T-DV2
Standard-cell library	SkyWater 130 nm `sky130_fd_sc_hd`	SAED32 educational kit (installed, see box below) or the team PDK	Provided by the coordinator in `lib/` (never committed)

SYNOPSYS CLOUD

The installation used by the team when this guide was written (versions change; `check_tools.sh` prints the ones you have): VCS and Verdi 2024.09, SpyGlass 2025.06, VC Static 2024.09 (VC SpyGlass Lint/CDC, VC Formal), Fusion Compiler 2025.06, PrimeTime 2025.06, Formality 2026.03, all under `/global/apps/<tool>/<version>/`; Verilator and GTKWave are under `/global/apps/freeware/`. Design Compiler is installed but *not licensed* for the team at the time of writing: synthesize with Fusion Compiler. The Synopsys SAED32 educational library kit (`.db`, NDM, technology file, Verilog cell models, SRAM macros) is installed under `/global/apps/lynxnxt/2024.09/fpm_FC_saed32/SAED32_data`; the flow uses it through `LIB_DB`, `LIB_NDM`, `LIB_TF` and `LIB_VERILOG`. The repository's `env.sh` (bash, zsh) and `env.csh` (csh, tcsh) detect these tools and set the default `SIM`, `LINT_TOOL`, `SYN_TOOL` and `STA_TOOL`; `make help` shows the values in use.

NOTE | Why the open-source flow stays in the labs.

The open-source tools are small enough to read: you can see every step Yosys runs and every line of an OpenSTA script. The commercial tools do the same steps with more options and better results. Learning the steps once, in the open, makes the commercial reports readable. Where the two tools disagree (for example Verilator is two-state and VCS is four-state, so an uninitialized register is 0 in one and X in the other), the lab points it out; explaining such a difference is part of the review.

9.2 Other Tools

Purpose	Tool	First used
Build and version control	GNU Make, Git	L0
Plotting and scripts	Python 3 (matplotlib, pandas)	P3, T-AM1
Software toolchain, ISA simulator	riscv-gnu-toolchain, Spike	Reading track, team projects

10 Weekly Study Requirements

10.1 Reading and Lectures

Read the assigned sections and watch the assigned lectures. For each lecture record the learning objectives, the important concepts, open questions and the connection to the current lab. Engineers are not expected to follow two complete courses in parallel.

10.2 Weekly Technical Summary

Commit a 1–2 page summary to `doc/reports/weekNN.md` that answers:

1. What did you learn this week?
2. Which concepts were difficult?
3. How does the concept appear in a real processor?
4. How is it implemented in RTL, verified, and what does it cost in area and timing?
5. What questions remain unanswered?

10.3 Weekly Review

The coordinator reviews the merge request of the current lab and asks the review questions listed at the end of each lab. The review emphasizes reasoning and hardware behavior rather than memorization. Every lab ends with a review that the engineer must pass before the lab is tagged as complete.

11 Submission Requirements

Each lab is developed on its own branch (for example `lab/r1-seq-mul`) and submitted as a merge request. Unless the lab states otherwise, a submission contains:

- The source files in the paths given by the lab (RTL, testbench, constraints, scripts).
- The block's `README.md` and the `doc/*.md` pages the lab asks for: how to reproduce every result with `make`, results, analysis, known limitations.

- Logs and reports referenced by the documentation (small text files only; everything else stays in `build/`).
- A clean `make check-all` on a fresh clone.

Screenshots without explanation, or code copied without understanding, are not accepted. After the review the coordinator merges the branch and creates the completion tag (`r1-done`).

12 Assessment

Each lab has its own weighted assessment table. Across the program the weights follow this pattern:

Criterion	Typical weight
Functional correctness of the deliverable	25–35%
Quality of the main skill of the phase (RTL quality, verification quality, analysis quality)	25–35%
Experiments and their explanation	10–20%
Flow reuse and reproducibility	5–10%
Documentation and technical explanation	10–15%

A result that passes a few tests is not complete. The engineer must show which corner cases were considered, which checks prove them, and which risks remain.

13 Completion Criteria

The **common core** is complete when the engineer has:

- completed and passed the reviews of L0, R1–R4, V1–V3 and P1–P3;
- four blocks that lint cleanly, pass their regressions with functional coverage closed, synthesize without latches, pass gate-level simulation and have documented area and timing;
- submitted the weekly summaries.

A **track** is complete when its labs are reviewed and the engineer has given a 30-minute technical presentation of the track results to the team.

Completing the program is not a certification that an engineer can independently implement advanced processor features. Readiness for production tasks is decided through supervised project work.

14 Final Objective

By the end of the onboarding, every engineer can explain, for a block they built themselves:

- why it is organized the way it is, and what each flip-flop is for;
- how it was verified, what the checks prove, and what they do not;
- what it costs in area, what limits its frequency, and how the RTL would change to improve it;
- how the block's behavior and performance would be predicted by a model before the RTL exists.

Design the block.
Verify the block.
Implement the block.
Then specialize.